Improving SSH's security with SSHFP DNS records

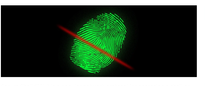
By Sebastian Neef on 2 Dec 2022

Category: Tech matters

Tags: DNS, DNSSEC, Guest Post, measurement

3 Comments

Blog home



Secure Shell (SSH) is a popular network protocol for accessing a shell on (remote) systems. Users might not pay enough attention to detail when asked to verify the system's authenticity on the first connection to a system, thereby risking a network-level security breach.

In this article, I will explain how SSHFP DNS records can help mitigate such risks and share the results of our large-scale analysis. At the time of measurement, only 1 in 10,000 domains used SSHFP records and less than 50% used a correct configuration.

My colleague and I from Technische Universität Berlin conducted some research on the SSHFP record type and its usage and security in early 2022. The resulting paper was presented at CANS 2022 and published soon after. I also provided a little weak-peak at DNS-OARC 39. All code and data are already public to allow for further research.

SSH host key verification

Before we dive into our analysis and results, we need to understand what host key verification is and why it is important. When connecting to an SSH for the very first time, the OpenSSH client will ask the user to verify the authenticity of the server as seen in Figure 1.

There exists anecdotal evidence that many users just blindly answer 'Yes' without considering potential security risks. A network-based attacker might be able to perform malice-in-the-middle (MITM) attacks leading to credentials leaks or session-hijacking.

```
$ ssh server
The authenticity of host 'server (192.168.10.24)' can't be established.
ECDSA key fingerprint is SHA256:jqlVVvG...hDc.
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

Figure 1 — SSH asks the user to verify the authenticity of a server by comparing the host key fingerprint on the first connection.

There are multiple methods for the host key fingerprint verification according to the SSH standard (RFC 4251):

- Manually, by the user
- Automated, via DNS
- Automated, via CA

Manual process

With the manual process, the user needs to compare the server's host key fingerprint (such as SHA256:jqlVVvG...hDc in Figure 1) against an out-of-band obtained fingerprint. This usually requires contacting the administrator responsible for the server.

Furthermore, this approach entails the risk of human error when comparing fingerprints.

Automated, via CA

RFC 4251 also touches on the possibility of using a certificate authority to verify host key fingerprints. However, this approach requires the deployment of a root-CA key to all user devices and signing all host keys with them. The former might not be achievable or suitable in certain scenarios and we, therefore, disregard this verification method.

Automated, via DNS

Another approach that we analysed in more detail is the distribution of the host key fingerprints via a special resource record. This removes any user interaction, potential human error, or presumed configuration of a user's device because OpenSSH-client can query the DNS and do the comparison. It only requires the administrator to publish a host's fingerprints as a set of SSHFP records, which can be securely obtained (DNSSEC). If the latter holds, MITM attacks are mitigated, and overall security is improved.

SSHFP DNS records

RFC 4255 defines the SSHFP's resource record schema. In representation format it looks like this:

```
SSHFP <KEY-ALGO> <HASH-TYPE> <FINGERPRINT>
```

The KEY-ALGO is a numeral representing the key's algorithm (Table 1), whereas HASH-TYPE defines the used hash algorithm (Table 2). The fingerprint is the hash's hexadecimal digest.

KEY-ALGO	Algorithm
0	reserved
1	RSA
2	DSA
3	ECDSA
4	ED25519
5	unassigned
6	ED448

Table 1 — Values for the SSHFP KEY-ALGO field.

HASH-TYPE	Algorithm
0	reserved
1	SHA1
2	SHA256

Table 2 — Values for the SSHFP HASH-TYPE field.

In the real world, the records will look like this:

```
$ dig SSHFP someserver.tld +noall +answer

; <<>> DiG 9.11.4-P2-RedHat-9.11.4-26.P2.el7_9.10 <<>> SSHFP someser
;; global options: +cmd
someserver.tld.     3600    IN    SSHFP    1 1 89F6A81021757A2
someserver.tld.     3600    IN    SSHFP    1 2 415BF2E19212D8A0
someserver.tld.     3600    IN    SSHFP    3 1 931CA0B07B7D2C6
someserver.tld.     3600    IN    SSHFP    3 2 655AC815F1A9B0E
someserver.tld.     3600    IN    SSHFP    4 1 F45DA8047B119BD
someserver.tld.     3600    IN    SSHFP    4 2 C6DE21BF23A123b
```

As can be seen in the listing, `someserver.tld` has three different keys (RSA, ECDSA, ED25519) and a SHA1/SHA256 fingerprint for each of them. These records can also easily be generated using OpenSSH's built-in tools:

```
$ ssh-keyscan -D someserver.tld
; someserver.tld:22 SSH-2.0-OpenSSH_8.9p1 Ubuntu-3
someserver.tld IN SSHFP 1 1 89F6A81021757A2b5b0b7c72c44c404068
someserver.tld IN SSHFP 1 2 415BF2E19212D8A0205A8121ccf5ced79e15f
; someserver.tld:22 SSH-2.0-OpenSSH_8.9p1 Ubuntu-3
someserver.tld IN SSHFP 3 1 931ca0B07B7d2c6L6ZF9Fda1326f13ce57bcd
someserver.tld IN SSHFP 3 2 655AC815F1a9B0e2d0Bcef153B05572bc234
; someserver.tld:22 SSH-2.0-OpenSSH_8.9p1 Ubuntu-3
someserver.tld IN SSHFP 4 1 F45DA8047B119BD5Jb31c5a1cf2f8918b63931
someserver.tld IN SSHFP 4 2 c6de21bF23a123b92d4b9e04ea71dc18bd8f72f
; someserver.tld:22 SSH-2.0-OpenSSH_8.9p1 Ubuntu-3
; someserver.tld:22 SSH-2.0-OpenSSH_8.9p1 Ubuntu-3
```

So, we believe that deploying host key verification fingerprints in the DNS is almost trivial. Also, an update is only required should the host keys change. However, the SSHFP records must reach the client in a secure fashion, otherwise, the MITM could happen on the DNS instead of the SSH layer.

Finally, OpenSSH-client needs to be told to use DNS for fingerprint verification using the `-o VerifyHostKeyDNS=yes` option. Although this feature has been implemented in OpenSSH for many years, it is unfortunately still not enabled by default. The debug output tells us what will happen in the background:

```
$ ssh -o UserKnownHostsFile=/dev/null -o VerifyHostKeyDNS=yes -v ser
[...]
debug1: Connecting to someserver.tld [IP.IP.IP.IP] port 22.
debug1: Connection established.
[...]
debug1: Server host key: ssh-ed25519 SHA256:rXdhPIEdjaRlJ6G6xHcGLj
debug1: found 6 secure fingerprints in DNS
debug1: verify_host_key_dns: matched SSHFP type 4 fptype 1
debug1: verify_host_key_dns: matched SSHFP type 4 fptype 2
debug1: matching host key fingerprint found in DNS
[...]
```

As can be seen, OpenSSH establishes a connection to the server, obtains the server-side host key fingerprint, then queries the domain for SSHFP records and compares them. It confirms that the fingerprints are secure and matching, thus continuing with the connection.

Large-scale analysis

In order to measure the prevalence of SSHFP records on the Internet, we first built a measurement system that works as follows:

1. Query a domain for SSHFP records.
2. If SSHFP records exist, query the domain for their A records.
3. If A records exist, try to obtain the server-side host key fingerprints using ssh-keyscan.
4. Compare SSHFP fingerprints and server-side fingerprints to find (mis)matches.
5. If matches exist, repeat the SSHFP query, but with a DNSSEC-validating resolver.

We ran this analysis on two different sets of domains:

- Tranco 1M domain list
- >51M domains from certificate transparency logs over the course of 26 days.

Our paper dives deep into the details of the results, but the most interesting parts in my opinion are:

- Only 1 in 10,000 domains have an SSHFP record set.
- Less than 50% of the records are secured with DNSSEC.
- In many cases not all KEY-ALGO and HASH-TYPE combinations exist as SSHFP records (or are outdated), thus leaving it to chance whether a DNS-based verification is successful or a fallback to other methods (that is, manual verification) is done.

Although there is room for improvement when looking at the adoption of SSHFP records, a 50% mark of secured records is unfortunate, because in such setups there is no security benefit, as the MITM-attack shifts from the SSH layer to the DNS layer.

There might be a 'chicken-and-egg' situation between SSHFP and DNSSEC here — one requires the other, but the other is not as widely adopted as it could be. However, internal usage of SSHFP records is not measurable from the outside, so the actual number might be somewhat larger.

Nonetheless, we believe that proper deployment and usage of SSHFP records could provide security benefits at little cost in many situations.

Here's a three-point action plan if you want to start using SSHFP records:

1. Enable DNSSEC for your domain(s).
2. Collect and deploy the SSHFP records for all your SSH-accessible systems using ssh-keyscan.
3. Add `VerifyHostKeyDNS=yes` to your `~/.ssh/config`.

If you are interested in more details and in-depth analysis, have a look at our paper.

Sebastian Neef is a PhD candidate and Research Assistant at the Technische Universität Berlin's 'Chair for Security in Telecommunications', with a focus on network and web security.

This post is based on the accepted and soon-to-be-published paper "Oh SSH-it, what's my Fingerprint? A Large-Scale Analysis of SSH Host Key Fingerprint Verification Records in the DNS" (Sebastian Neef and Nils Wisiol, prepared).

Continue the conversation on this topic at Orbit.

The views expressed by the authors of this blog are their own and do not necessarily reflect the views of APNIC. Please note a Code of Conduct applies to this blog.

3 Comments

Joseph Ottinger
December 5, 2022 at 11:00 pm

Note that not all DNS providers SUPPORT SSHFP as a record type. Namecheap, for example, does not, so if you're using Namecheap, your ability to add SSHFP is nonexistent, as far as I can tell.

Reply ↓

James Marr
December 11, 2022 at 5:04 am

I was going to post the same thing — AWS Route53 doesn't support SSHFP either.

One thing I've recently wanted to try out is to stuff the record in a TXT record under the prefix "_sshfp." and then make use of the KnownHostsCommand config option to look up the data (and potentially enforce dns-sec validation)

Reply ↓

Patryk Krawaczynski
February 22, 2024 at 6:46 am

man-in-the-middle not malice-in-the-middle.

Reply ↓

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name *

Email *

Save my name and email in this browser for the next time I comment.

Yes, add me to your mailing list

进行人机身份验证 reCAPTCHA 隐私 - 条款

Notify me of follow-up comments via email. You can also subscribe without commenting.

Post Comment

Top

Get Updates

Email *

Subscribe! Show options

Authors

- Adli Wahid
- Aftab Siddiqui
- Geoff Huston
- George Michaelson
- Jen Linkova
- Jia Rong Low
- Job Snijders
- Kathleen Moriarty
- Ulrich Speidel
- Vitaly Ramluk

Show All (998)

Tags

Related Articles

Abuse of dangling DNS records on cloud platforms

by Haya Schulmann April 4, 2024 Guest Post: Real-life hijacks of cloud resources identified for the first time.

Journeys in hosting 1/4 — Precomputed SSH host keys

by John Kristoff December 18, 2025 Guest Post: An investigation into LightNode's curious use of precomputed SSH host keys — a small operational quirk with security implications.

A first look at suspicious IRR records

by Ben Du May 1, 2024 Guest Post: Proposing and validating a workflow to identify suspicious IRR records.

Use of HTTPS resource records

by Jan Schaumann December 18, 2023 Guest Post: Measuring HTTPS resource record adoption in the wild.